

NSI - Numérique et Sciences Informatiques

Th4Ch5 - TD1 - Les processus

Notion de processus

Définition

Un programme écrit à l'aide d'un langage de haut de niveau (on parle de "code source") est, comme nous l'avons vu l'année dernière, transformé en langage machine afin de pouvoir être exécuté par un ordinateur.

On appelle **processus** un programme **en cours d'exécution**.

Attention ! il ne faut pas confondre le code source du programme et un processus, qui lui correspond à l'exécution de ce programme par un ordinateur. Pour prendre une image assez classique, si une recette de cuisine correspond au code source du programme, le cuisinier en train de préparer cette recette dans sa cuisine correspond à un processus.

Création d'un processus

Un processus peut **créer** un ou plusieurs processus à l'aide d'une **commande système** (**fork** sous les systèmes de type Unix). Imaginons un processus A qui crée un processus B. On dira que A est le **père** de B et que B est le **fils** de A. B peut, à son tour créer un processus C (B sera le **père** de C et C le **fils** de B). On peut modéliser ces **relations père/fils** par une **structure arborescente**.

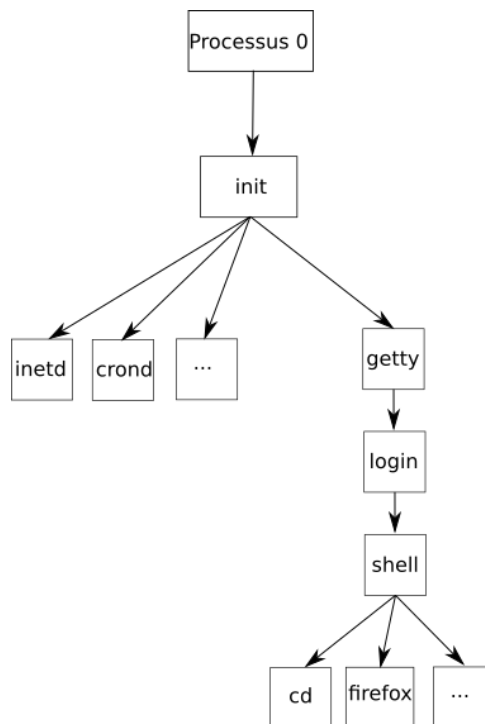
Si un processus est créé à partir d'un autre processus, **comment est créé le tout premier processus ?**

Dans un système d'exploitation comme Linux, **au moment du démarrage de l'ordinateur** un tout premier processus (appelé processus 0 ou encore Swapper) est créé à partir de "rien" (il n'est le

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	1/8

fil d'aucun processus). Ensuite, ce **processus 0** crée un processus souvent appelé **init** ("init" est donc le fils du processus 0). À partir de **init**, les processus nécessaires au bon fonctionnement du système sont créés (par exemple les processus **crond**, **inetd**, **getty**,...). Puis d'autres processus sont créés à partir des fils de **init**...

On peut résumer tout cela avec le schéma suivant :



N. B. Tous ces noms de processus ne sont pas à retenir, ils sont juste donnés pour l'exemple. Il est juste nécessaire d'avoir compris les notions de processus père et processus fils et la structure arborescente.

PID et PPID

Chaque processus possède un identifiant appelé **PID (Process Identifier)**, ce PID est un nombre. Le premier processus créé au démarrage du système a pour PID 0, le second 1, le troisième 2... Le système d'exploitation utilise un compteur qui est incrémenté de 1 à chaque création de processus, le système utilise ce compteur pour attribuer les PID aux processus.

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	2/8

Chaque processus possède aussi un PPID (Parent Process Identification). Ce PPID permet de connaître le processus parent d'un processus (par exemple le processus "init" vu ci-dessus à un PID de 1 et un PPID de 0). À noter que le processus 0 ne possède pas de PPID (c'est le seul dans cette situation).

À faire vous-même 1

En vous basant sur le schéma ci-dessus, donnez le PID (en partant du principe qu'il est créé juste après init) et le PPID du processus "getty".

À faire vous-même 2

1. Démarrez le navigateur Web (firefox) de votre machine.
2. Après avoir ouvert un terminal, tapez la commande suivante :

```
ps -aef
```

Vous devriez avoir plusieurs informations sur les processus en cours sur votre ordinateur, notamment les **PID** et les **PPID** de ces processus. Comme il y a beaucoup de processus, vous allez **filtrer** le résultat de la commande, c'est à dire trier les lignes du résultat pour ne conserver que les lignes contenant le mot **firefox**. Pour cela vous allez utiliser la commande **grep**.

3. Pour filtrer tapez la commande suivante :

```
ps -aef | grep firefox
```

4. A partir des PID et des PPID des processus, construisez l'arborescence des processus affichés.

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	3/8

États d'un processus

Tous les systèmes d'exploitation "modernes" (Linux, Windows, macOS, Android, iOS...) sont capables de gérer l'exécution de plusieurs processus en même temps. Mais pour être précis, cela n'est pas en véritable "en même temps", mais plutôt un "chacun son tour". Pour gérer ce "chacun son tour", les systèmes d'exploitation attribuent des "états" au processus.

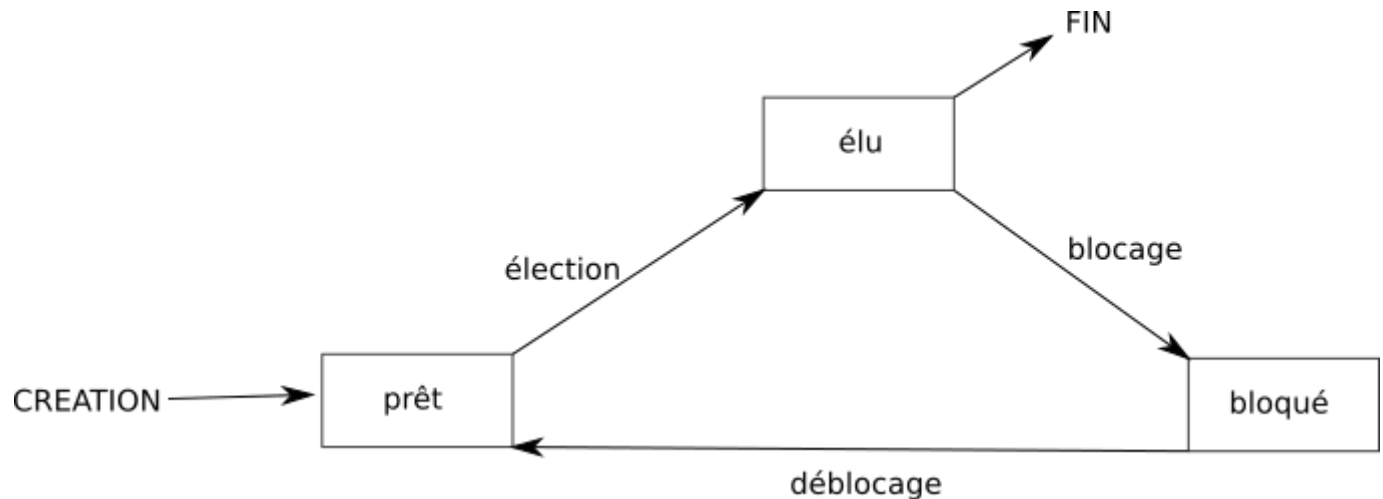
Voici les différents états :

- **Élu** : Lorsqu'un processus est en train de s'exécuter (qu'il utilise le microprocesseur), on dit que le processus est dans l'état **élu**.
- **Bloqué** : Un processus qui se trouve dans l'état élu peut demander à accéder à une ressource pas forcément disponible instantanément (par exemple lire une donnée sur le disque dur). Le processus ne peut pas poursuivre son exécution tant qu'il n'a pas obtenu cette ressource. En attendant de recevoir cette ressource, il passe de l'état **élu** à l'état **bloqué**.
- **Prêt** : Lorsque le processus finit par obtenir la ressource attendue, celui-ci peut potentiellement reprendre son exécution. Mais comme nous l'avons vu ci-dessus, les systèmes d'exploitation permettent de gérer plusieurs processus "en même temps", mais un seul processus peut se trouver dans un état **élu** (le microprocesseur ne peut "s'occuper" que d'un seul processus à la fois). Quand un processus passe d'un état **élu** à un état **bloqué**, un autre processus peut alors "prendre sa place" et passer dans l'état **élu**. Le processus qui vient de recevoir la ressource attendue ne va donc pas forcément pouvoir reprendre son exécution tout de suite, car pendant qu'il était dans un état **bloqué** un autre processus a "pris sa place". Un processus qui quitte l'état **bloqué** ne repasse pas forcément à l'état **élu**, il peut, en attendant que "la place se libère" passer dans l'état **prêt** (sous entendu "j'ai obtenu ce que j'attendais, je suis prêt à reprendre mon exécution dès que la "place sera libérée").

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	4/8

Le passage de l'état "prêt" vers l'état "élu" constitue l'opération "d'élection". Le passage de l'état élu vers l'état bloqué est l'opération de "blocage". Un processus est toujours créé dans l'état "prêt". Pour se terminer, un processus doit obligatoirement se trouver dans l'état "élu".

On peut résumer tout cela avec le diagramme suivant :



Il est vraiment important de bien comprendre que le "chef d'orchestre" qui attribue aux processus leur état **élu**, **bloqué** ou **prêt** est le système d'exploitation. On dit que le système gère l'**ordonnancement** des processus (tel processus sera prioritaire sur tel autre...)

Chose aussi à ne pas perdre de vue : un **processus** qui utilise une **ressource R** doit la **libérer** une fois qu'il a fini de l'utiliser afin de la rendre disponible pour les autres processus. Pour libérer une **ressource**, un processus doit obligatoirement être dans un état **élu**.

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale		Page 5/8

Observer les processus

Sous Linux il existe des commandes permettant de visualiser les processus. Vous allez maintenant observer les processus de votre machine.

À faire vous-même 3

1. Après avoir ouvert un terminal, tapez la commande suivante :

ps -aef

2. Vous devriez avoir plusieurs informations sur les processus en cours sur votre ordinateur, notamment les **PID** et les **PPID** de ces processus.
3. Identifiez le programme correspondant au premier processus.

La commande **ps** ne permet pas de suivre en temps réel les processus (affichage figé). Pour avoir un suivi en temps réel, vous pouvez utiliser la commande **top**.

À faire vous-même 3

Après avoir ouvert un terminal, tapez la commande suivante :

top

Pour en savoir plus sur la commande **top**, consultez la page <http://debian-facile.org/doc:systeme:top>

À faire vous-même 4

1. Démarrez le **navigateur Web** de votre machine virtuelle.
2. En utilisant la commande **top** dans un terminal, observez ce qui se passe au niveau des processus quand vous ouvrez ou supprimez un onglet dans votre **navigateur Web**.

Il est possible de supprimer un processus en utilisant la commande **kill**. L'utilisation de cette commande est très simple, il suffit de taper **kill** suivi du **PID** du processus à supprimer (exemple : **kill 4242** permet de supprimer le processus de **PID 4242**)

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	6/8

À faire vous-même 5

- Ouvrez 2 terminaux, placez-les l'un à côté de l'autre. Dans l'un des 2 terminaux, exécutez la commande **top**

Vous devriez obtenir quelque chose qui ressemble à ceci :

```

david@ordi-fixe: ~
top - 11:53:40 up 2:36, 1 user, load average: 0.03, 1.00, 1.29
Tasks: 236 total, 1 en cours, 185 en veille, 0 arrêté, 0 zombie
Ucste(s): 10.6 ut, 3.2 sy, 0.0 ni, 03.9 id, 2.1 wa, 0.0 hi, 0.2 si, 0.0 st
Mem Mem : 8874136 total, 582524 lib, 2292196 util, 5199416 temp/cache
Mem Mem : 2097148 total, 2896360 lib, 760 util, 4795096 dispo Mem

#PID UTIL. PR NI VIRT RES SHR S %CPU %MEM TIME+ COM.
1311 david 20 # 3411360 144692 74608 S 21.0 1.0 0:17.59 cinnamon
889 root 20 # 562956 95336 67724 S 11.6 1.2 4:32.40 xorg
6545 david 20 # 2451760 112804 74812 S 5.3 1.4 2:30.64 splayer
11888 david 20 # 546488 33568 26444 S 4.6 0.4 8:00.48 gnome-scre+
6824 david 20 # 867952 263240 137072 S 4.3 3.3 6:17.45 chrome
6836 david 20 # 84.842g 36144 28848 S 4.8 0.4 3:24.72 nacl_helper
7824 david 20 # 729548 101156 71624 S 3.0 1.3 0:02.89 chrome
1185 david 9 -11 2128388 17732 13584 S 2.8 0.2 1:13.29 pulseaudio
5127 david 20 # 1815864 294644 135704 S 1.7 3.6 3:04.00 atox
6815 david 20 # 42360 4220 3528 R 2.3 0.1 0:15.45 top
6756 david 20 # 715132 98164 74648 S 1.8 1.2 0:41.26 chrome
0 root 20 # 0 0 0 S 0.3 0.0 0:17.74 rcu_sched
1174 david 20 # 188840 5120 4556 S 0.3 0.1 0:00.20 dcourt-very+
1387 david 20 # 755332 98836 38108 S 0.3 1.2 0:10.36 cinnamon-s+
7370 david 20 # 594152 30868 27504 S 0.3 0.5 0:06.00 gnome-term+
0882 root 20 # 0 0 0 S 0.3 0.0 0:01.03 kworker/u8+
9749 root 20 # 0 0 0 S 0.3 0.0 0:00.00 kworker/1:1
  
```

- Fermez votre **navigateur Web** et observez le résultat dans le terminal exécutant **top**.
- Ouvrez votre **navigateur Web** et observez le résultat dans le terminal exécutant **top**. Selon le navigateur que vous utilisez, il se peut que le fonctionnement du navigateur soit associé à plusieurs processus.
- Notez le **PID** des processus liés au fonctionnement du navigateur.
- Utilisez la commande **kill** afin de supprimer le (ou les) processus lié(s) au fonctionnement du navigateur. Que se passe-t-il ?

Interblocage

Pour terminer, nous allons maintenant étudier le phénomène d'interblocage (deadlock en anglais). Soit 2 processus P1 et P2, soit 2 ressources R1 et R2. Initialement, les 2 ressources sont "libres" (utilisées par aucun processus). Le processus P1 commence son exécution (état élu), il demande la ressource R1. Il obtient satisfaction puisque R1 est libre, P1 est donc dans l'état "prêt". Pendant ce temps, le système a passé P2 à l'état élu : P2 commence son exécution et demande la ressource

Auteur(s)	David Roche (www.pixees.fr) B. Dutilloy	NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale
		Page		7/8

R2. Il obtient immédiatement R2 puisque cette ressource était libre. P2 repasse immédiatement à l'état élu et poursuit son exécution (P1 lui est toujours dans l'état prêt). P2 demande la ressource R1, il se retrouve dans un état bloqué puisque la ressource R1 a été attribuée à P1 : P1 est dans l'état prêt, il n'a pas eu l'occasion de libérer la ressource R1 puisqu'il n'a pas eu l'occasion d'utiliser R1 (pour utiliser R1, P1 doit être dans l'état élu). P2 étant bloqué (en attente de R1), le système passe P1 dans l'état élu et avant de libérer R1, il demande à utiliser R2. Problème : R2 n'a pas encore été libéré par P2, R2 n'est donc pas disponible, P1 se retrouve bloqué.

Résumons la situation à cet instant : P1 possède la ressource R1 et se trouve dans l'état bloqué (attente de R2), P2 possède la ressource R2 et se trouve dans l'état bloqué (attente de R1)

Pour que P1 puisse poursuivre son exécution, il faut que P2 libère la ressource R2, mais P2 ne peut pas poursuivre son exécution (et donc libérer R2) puisqu'il est bloqué dans l'attente de R1. Pour que P2 puisse poursuivre son exécution, il faut que P1 libère la ressource R1, mais P1 ne peut pas poursuivre son exécution (et donc libérer R1) puisqu'il est bloqué dans l'attente de R2. Bref, la situation est totalement bloquée !

Cette situation est qualifiée d'interblocage (deadlock en anglais).

Il existe plusieurs solutions permettant soit de mettre fin à un interblocage (cela passe par l'arrêt d'un des 2 processus fautifs) ou d'éviter les interblocage, mais ces solutions ne seront pas étudiées ici.

À faire vous-même 6

Mettez au point une petite saynète de théâtre permettant d'expliquer l'interblocage de 2 processus. Par exemple on pourra avoir 2 actrices ou acteurs qui joueront le rôle des processus, des étiquettes qui représenteront les différents états (élu, bloqué ou prêt) des processus et des étiquettes qui représenteront les ressources R1 et R2.

Auteur(s)		David Roche (www.pixees.fr) B. Dutilloy		NSI - Architectures matérielles, Systèmes d'Exploitation et Réseaux		
Rév	1	Date	04/04/22	Terminale	Page	8/8